Document number: P0206R0

Ville Voutilainen
2016-01-27

Discussion about std::thread and RAII

Abstract

C++ continues not to provide a thread type that would join() automatically on scope exit. This causes exception-safety problems, because failing to join() in all code paths causes the destructor of a std::thread to terminate(). This paper explores various ways to solve the problem.

Contents

The problem, reiterated

Herb Sutter provided the following code example:

```
std::vector<std::pair<unsigned int, unsigned int>> partitions =
  utils::partition_indexes(0, size-1, num_threads);
std::vector<std::thread> threads;

LOG(LOG_DEBUG, "controller::reload_all: starting reload threads...");
for (unsigned int i=0; i<num_threads-1; i++) {
  threads.push_back(std::thread(reloadrangethread(this,
  partitions[i].first, partitions[i].second, size, unattended)));
}

LOG(LOG_DEBUG, "controller::reload_all: starting my own reload...");
this->reload_range(partitions[num_threads-1].first,
  partitions[num_threads-1].second, size, unattended);

LOG(LOG_DEBUG, "controller::reload_all: joining other threads...");
for (size_t i=0; i<threads.size(); i++) {
  threads[i].join();
}
```

The problem with the code is that the push_back can fail with an exception, and in general, any exception thrown will skip the loop that joins all threads, and any unjoined threads destroyed with the destruction of the thread vector will cause the program to terminate.

The status quo solution

The way to make the code exception-safe is to use a third-party thread wrapper that joins the thread on destruction. That is, instead of containing threads, the vector should hold such wrappers. Scott Meyers, Anthony Williams and Bjarne Stroustrup all describe such thread wrappers in their most recent books.

The problem with the status quo solution

It's inconvenient to need to use a third-party wrapper for something as fundamental as exception-safety of using std::thread.

There are multiple such third-party wrappers, and it takes more effort to take one of them into use than using a solution that would already be provided by the standard library implementation. The following sections enumerate some potential solutions.

Solution 1: Change std::thread::~thread to auto-join

This solution has been previously proposed by Herb Sutter in n3636. That paper contains proposed wording for the necessary changes to std::thread. Implementation is trivially easy for standard library vendors. As an addition, it's suggested that for audiences willing to use the current terminating semantics, a new type should be added that will not join on destructor, but will terminate instead.

The opposition to this solution has been that it changes the semantics of existing code; there apparently are audiences who prefer the destruction of joinable threads (which can be seen a logic error) to terminate so that the program exits quickly, and external facilities can recover without the program waiting (or hanging on) a join().

The benefit of this solution is that it's easy to use as a default. No external wrapper needs to be taken into use, and it's arguably a better default that caters better to non-expert use cases. Experts that want termination need to use an alternative approach, rather than non-experts who do not even realize their code isn't exception-safe needing to use an alternative approach.

Solution 2: Add a new thread type that auto-joins

As opposed to adding a thread wrapper (see later), it's been suggested that it would be better to add a new thread type that has the full API of std::thread, and recommend using this new thread type instead of std::thread. Following such advise avoids the problems that would arise when using a thread combined with a thread wrapper; forgetting to wrap would make the exception-safety problems arise again.

The benefit of this solution is that it's non-intrusive; all existing users of std::thread are unaffected, for better or worse. Replacing the uses of std::thread with the uses of the new type where need be is arguably straightforward.

The downside of this solution is that it introduces another thread type that users will need to consider in addition to std::thread.

Solution 3: Add a thread wrapper that auto-joins

The aforementioned book authors all provide a thread wrapper that joins in its destructor. The wrapper as such can be made fairly simple, and doesn't necessarily need to duplicate the full API of std::thread. It's non-intrusive for existing code that uses std::thread, and can be applied as necessary. It also doesn't need to duplicate all of the functionality in std::thread.

The benefit of this solution is that it's non-intrusive; all existing users of std::thread are unaffected, for better or worse. The wrapper can be relatively simple, and can be specification-wise and implementation-wise a fairly lean solution. Replacing the uses of std::thread with the uses of the new type where need be can be straightforward.

The downside of this solution is that remembering to use the wrapper is tedious. Another downside is that if it's really a wrapper, ownership questions arise; and if the wrapper owns the underlying std::thread, then there's no real difference between a new thread type and such a wrapper. Thus the 'wrapper' is NOT proposed with wording; the proposed wording for solutions for 1 and 2 ARE such wrappers; they own a thread, and are convertible from and to a thread.

Solution summary

Solution 1 and Solution 2 are the same option from the point of view that they both say "add a thread type that auto-joins." The only difference between Solution 1 and Solution 2 is which type gets the std::thread name - the one that has the current behavior, or the one that auto-joins. Also, both solutions provide a "wrapper", as the new thread types proposed in those solutions are convertible from and to std::thread.

Wording for Solution 1

This solution has been previously proposed by Herb Sutter in n3636. That paper contains proposed wording for the necessary changes to std::thread, so this paper will not repeat that part of the wording. As an addition, this paper will provide wording for a new thread type should be added that will not join in destructor or assignment operator, but will terminate instead, thus providing a migration solution for users who want to retain the C++11 semantics that std::thread previously had.

In [thread.threads]/1, insert as follows:

```
Header <thread> synopsis
  namespace std {
    class thread;
```

```
    class basic_thread;
```

After [thread.thread.this], add a new section as follows:

```
Class basic_thread [thread.basic_thread.class]

namespace std {
  class basic_thread {
    public:
    // types:
    class id;
    typedef implementation-defined native_handle_type; // See 30.2.3
    // construct/copy/destroy:
    basic_thread() noexcept;
    template <class F, class ...Args> explicit basic_thread(F&& f, Args&&... args);
    ~basic_thread() noexcept; // semantics different from std::thread
    basic_thread(const basic_thread&) = delete;
    basic_thread(basic_thread&&) noexcept;
    explicit basic_thread(thread&& x) noexcept; // addition to std::thread interface
    basic_thread& operator=(const basic_thread&) = delete;
    basic_thread& operator=(basic_thread&&) noexcept; // semantics different from std::thread
    basic_thread& operator=(thread&& x) noexcept; // addition to std::thread interface
    // members:
    void swap(basic_thread&) noexcept;
    bool joinable() const noexcept;
    void join();
    void detach();
    id get_id() const noexcept;
    native_handle_type native_handle(); // See 30.2.
    thread to_thread(); // addition to std::thread interface
    // static members:
    static unsigned hardware_concurrency() noexcept;
  };
}


The class basic_thread provides the same facilities as thread, and
has the same members and the same semantics, with the differences
as described below.

basic_thread constructors [thread.basic_thread.constr]

explicit basic_thread(thread&& x) noexcept;

  Effects: Constructs an object of type basic_thread from x, and
  sets x to a default constructed state.

  Postconditions: x.get_id() == id() and get_id() returns the value
  of x.get_id() prior to the start of construction.

basic_thread destructor [thread.basic_thread.destr]

~basic_thread();

  Effects: If joinable(), calls std::terminate(). Otherwise, has no effects.
  [ Note: Either implicitly detaching or joining a joinable() thread in its
  destructor could result in difficult to debug correctness (for detach)
  or performance (for join) bugs encountered only when an exception is raised.
  Thus the programmer must ensure that the destructor is never executed while
  the thread is still joinable. -end note ]

basic_thread assignment [thread.basic_thread.assign]

basic_thread& operator=(basic_thread&& x) noexcept;
  Effects: If joinable(), calls std::terminate(). Otherwise, assigns
  the state of x to *this and sets x to a default constructed state.

  Postconditions: x.get_id() == id() and get_id() returns the value
  of x.get_id() prior to the assignment.

  Returns: *this

basic_thread& operator=(thread&& x) noexcept;
  Effects: If joinable(), calls std::terminate(). Otherwise, assigns
  the state of x to *this and sets x to a default constructed state.

  Postconditions: x.get_id() == id() and get_id() returns the value
  of x.get_id() prior to the assignment.
```

```
  Returns: *this

basic_thread members [thread.basic_thread.member]

thread to_thread();

  Postconditions: *this is set to a default constructed state. The state
  of the returned object is the state *this had prior to calling this function.

  Returns: A thread object initialized from *this.
  [Drafting note: an alternative approach would be to publicly inherit
  basic_thread from thread, and let derived-to-base conversion handle
  this conversion.]
```

Wording for Solution 2

In [thread.threads]/1, insert as follows:

```
Header <thread> synopsis
  namespace std {
    class thread;
    class safe_thread;
```

After [thread.thread.this], add a new section as follows:

```
Class safe_thread [thread.safe_thread.class]

namespace std {
  class safe_thread {
    public:
    // types:
    class id;
    typedef implementation-defined native_handle_type; // See 30.2.3
    // construct/copy/destroy:
    safe_thread() noexcept;
    template <class F, class ...Args> explicit safe_thread(F&& f, Args&&... args);
    ~safe_thread() noexcept; // semantics different from std::thread
    safe_thread(const safe_thread&) = delete;
    safe_thread(safe_thread&&) noexcept;
    safe_thread(thread&& x) noexcept; // addition to std::thread interface
    safe_thread& operator=(const safe_thread&) = delete;
    safe_thread& operator=(safe_thread&&) noexcept; // semantics different from std::thread
    safe_thread& operator=(thread&& x) noexcept; // addition to std::thread interface
    // members:
    void swap(safe_thread&) noexcept;
    bool joinable() const noexcept;
    void join();
    void detach();
    id get_id() const noexcept;
    native_handle_type native_handle(); // See 30.2.3
    thread to_thread(); // addition to std::thread interface
    // static members:
    static unsigned hardware_concurrency() noexcept;
  };
}



The class safe_thread provides the same facilities as thread, and
has the same members and the same semantics, with the differences
as described below.

safe_thread constructors [thread.safe_thread.constr]

safe_thread(thread&& x) noexcept;

  Effects: Constructs an object of type safe_thread from x, and
  sets x to a default constructed state.

  Postconditions: x.get_id() == id() and get_id() returns the value
  of x.get_id() prior to the start of construction.

safe_thread destructor [thread.safe_thread.destr]

~safe_thread() noexcept;

  Effects: If joinable(), calls join(). Otherwise, has no effects.
```

```
  [Note: Because ~safe_thread is required to be noexcept,
  if join() throws then std::terminate() will be called.
  --end note]

safe_thread assignment [thread.safe_thread.assign]

safe_thread& operator=(safe_thread&& x) noexcept;
  Effects: If joinable(), calls join(). Then, assigns
  the state of x to *this and sets x to a default constructed state.
  [Note: If join() throws then std::terminate() will be called.
  --end note]
  Postconditions: x.get_id() == id() and get_id() returns the value
  of x.get_id() prior to the assignment.

  Returns: *this

safe_thread& operator=(thread&& x) noexcept;
  Effects: If joinable(), calls join(). Then, assigns
  the state of x to *this and sets x to a default constructed state.
  [Note: If join() throws then std::terminate() will be called.
  --end note]
  Postconditions: x.get_id() == id() and get_id() returns the value
  of x.get_id() prior to the assignment.

  Returns: *this

safe_thread members [thread.safe_thread.member]

thread to_thread();

  Postconditions: *this is set to a default constructed state. The state
  of the returned object is the state *this had prior to calling this function.

  Returns: A thread object initialized from *this.
```